NAME :- Lokare Mahir Abdul Majeed
ROLL NO :- 23AI34
BATCH :- 02

EXPERIMENT NO :- 08

AIM :- To design and implement a Convolutional Neural Network (CNN) model for image classification using a standard dataset.

OBJECTIVE :-

1. To understand the concept of CNN for image classification.
2. To understand convolution and pooling operations.
3. To implement a CNN model using a standard image dataset.
4. To train the CNN model for multi-class image classification.
5. To evaluate the model using test accuracy and loss.
6. To visualize the classified images and training performance.

THEORY :-

1. Convolutional Neural Network :-

A Convolutional Neural Network (CNN) is a deep learning model mainly designed for processing and analyzing image data. CNNs automatically learn important visual features such as edges, textures, shapes and patterns through convolutional layers. They are widely used for image classification and other computer vision tasks.

Unlike a fully connected neural network, a CNN processes local regions of an image and maintains the spatial relationship between nearby pixels. As the data passes through successive layers, the network learns increasingly complex features that can be used to identify different objects.

For this experiment, the CIFAR-10 dataset is used. CIFAR-10 is a standard dataset for multi-class image classification containing colour images belonging to 10 different categories.

2. CIFAR-10 Dataset :-

CIFAR-10 consists of 60,000 colour images, each having a resolution of 32 × 32 pixels. The dataset contains 50,000 training images and 10,000 test images. Each image has three colour channels: Red, Green and Blue (RGB).

The ten classes of CIFAR-10 are: 0 – Airplane, 1 – Automobile, 2 – Bird, 3 – Cat, 4 – Deer, 5 – Dog, 6 – Frog, 7 – Horse, 8 – Ship, 9 – Truck.

Therefore, the input shape of an image is:

$$32 \times 32 \times 3$$

where 32 × 32 represents the image dimensions and 3 represents the RGB colour channels.

3. Main Components of CNN

A CNN generally consists of an input layer, convolutional layers, activation functions, pooling layers, flattening and fully connected layers.

Input Layer:

The input layer receives the image in the form of numerical pixel values. For CIFAR-10, every input image has the shape 32 × 32 × 3. The pixel values are normalized to the range 0 to 1 before training.

Convolutional Layer:

The convolutional layer applies learnable filters or kernels to the input image. These filters move across different regions of the image and perform mathematical operations to produce feature maps.

Different filters learn different visual features. The initial convolutional layers generally learn simple features such as edges and textures, while deeper layers can learn more complex patterns. A simplified convolution operation is represented as:

$$S(i, j) = \Sigma m \, \Sigma n \, I(i+m, j+n) \, K(m, n)$$

where I represents the input image and K represents the convolution kernel.

ReLU Activation:

The Rectified Linear Unit (ReLU) activation function is applied after convolution. It introduces non-linearity into the network and allows the CNN to learn complex patterns.

$$ReLU(x) = max(0, x)$$

Negative values are converted to zero while positive values are retained.

4. Pooling Layer

Pooling is used to reduce the spatial dimensions of feature maps while retaining important information. One commonly used pooling operation is Max Pooling.

In Max Pooling, the maximum value from a small region of the feature map is selected. A typical configuration is a 2 × 2 pooling window. Pooling helps reduce the computational requirements of the network and makes the extracted features less sensitive to small changes in the position of objects.

5. Flatten Layer

After the convolutional and pooling operations, the extracted feature maps are converted into a one-dimensional vector using the Flatten layer. The flattened representation contains the learned image features and is passed to the dense layers for classification.

6. Fully Connected Layer

The fully connected or dense layer combines the features extracted by the convolutional layers. It learns relationships between these features and uses them to distinguish between different image classes. In this experiment, a dense layer with 128 neurons and ReLU activation is used before the final classification layer.

7. Output Layer

Since CIFAR-10 contains ten classes, the final output layer contains 10 neurons. A Softmax activation function is used in the output layer. It converts the output values into probabilities for the ten possible classes. The class having the highest probability is selected as the predicted class. The Softmax function is given by:

$$Softmax(z_i) = e^{z_i} / \Sigma_j e^{z_j}$$

8. CNN Architecture for Image Classification

The convolutional layers perform feature extraction, pooling reduces the spatial dimensions, and the dense layers perform the final classification.

The CNN architecture used in this experiment consists of multiple layers that progressively extract and classify features from the input image. The Input Layer accepts a colour image of size 32 × 32 × 3. The first Convolutional Layer uses 32 filters of size 3 × 3 with ReLU activation to extract basic features such as edges and textures. A Max Pooling Layer with a 2 × 2 window then reduces the spatial dimensions of the feature maps. The second convolutional layer uses 64 filters to learn more detailed features, followed by another Max Pooling layer. The third convolutional layer uses 128 filters to extract higher-level features from the image. The resulting feature maps are then converted into a one-dimensional vector using the Flatten Layer and passed to a Dense Layer containing 128 neurons with ReLU activation. A Dropout Layer is used to reduce overfitting during training. Finally, the Output Layer contains 10 neurons with Softmax activation, corresponding to the ten CIFAR-10 classes, and produces the probability of the input image belonging to each class.

9. Working of CNN

The working process can be summarized as follows: the CIFAR-10 dataset is loaded; the pixel values of the images are normalized; the images are provided to the first convolutional layer; convolutional filters extract basic visual features; ReLU introduces non-linearity; Max Pooling reduces the spatial dimensions; further convolutional layers extract higher-level features; the Flatten layer converts the feature maps into a one-dimensional vector; the dense layer processes the extracted features; the Softmax layer produces probabilities for all ten classes; the class with the highest probability is selected as the prediction.

During training, the predicted class is compared with the actual class label. The loss is calculated and the optimizer updates the model weights using backpropagation. Repeating this process over multiple epochs allows the CNN to learn features useful for classification.

10. Training and Evaluation

The model is compiled using the Adam optimizer and sparse categorical cross-entropy loss function. Accuracy is used as the evaluation metric.

The training data is divided into training and validation portions. The training accuracy and validation accuracy can be plotted to observe the performance of the CNN during training. After training, the model is evaluated using the test dataset. Test loss and test accuracy are obtained to measure the classification performance on previously unseen images.

11. Applications

CNN-based image classification is used in several computer vision applications such as object recognition, image search, autonomous systems, medical image analysis and automated visual inspection.

CIFAR-10 is particularly useful for experimenting with CNN architectures because it provides a standardized multi-class image classification problem with RGB images and ten object categories.

PROCEDURE :-

1. Import TensorFlow and Matplotlib.
2. Load the CIFAR-10 image classification dataset.
3. Normalize the pixel values between 0 and 1.
4. Create the CNN model.
5. Add the first convolutional layer with ReLU activation.
6. Add a Max Pooling layer.
7. Add the second convolutional layer with ReLU activation.
8. Add another Max Pooling layer.
9. Add the third convolutional layer with ReLU activation.
10. Flatten the extracted feature maps.
11. Add a Dense layer with ReLU activation.
12. Add a Dropout layer.
13. Add a 10-neuron Softmax output layer.
14. Compile the model using Adam optimizer and sparse categorical cross-entropy.
15. Train and evaluate the CNN model.
16. Display sample predictions and training accuracy.

CODE :-

```
import tensorflow as tf

import matplotlib.pyplot as plt

from PIL import Image


(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()


x_train = x_train.astype("float32") / 255.0

x_test = x_test.astype("float32") / 255.0


class_names = [

    "Airplane",

    "Automobile",

    "Bird",

    "Cat",
```

```
    "Deer",
    "Dog",
    "Frog",
    "Horse",
    "Ship",
    "Truck"
]

model = tf.keras.Sequential([
    tf.keras.Input(shape=(32, 32, 3)),
    tf.keras.layers.Conv2D(
        32,
        (3, 3),
        activation="relu"
    ),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(
        128,
        (3, 3),
        activation="relu"
    ),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(
        128,
        activation="relu"
    ),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(
        10,
```

```
        activation="softmax"
    )
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)

test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print("Test Accuracy:", test_accuracy)

image_path = input("Enter image path: ")

image = Image.open(image_path).convert("RGB")
image = image.resize((32, 32))

image_array = tf.keras.utils.img_to_array(image)
image_array = image_array / 255.0

image_array = tf.expand_dims(
```

```
    image_array,
    axis=0
)

prediction = model.predict(
    image_array,
    verbose=0
)

predicted_class = prediction.argmax(axis=1)[0]

confidence = prediction[0][predicted_class] * 100

print(
    "Predicted Class:",
    class_names[predicted_class]
)

print(
    "Confidence:",
    confidence
)

plt.imshow(image)

plt.title(
    f"Predicted: {class_names[predicted_class]}"
)

plt.axis("off")
plt.show()
```

OUTPUT :-

```
Epoch 1/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 39s 61ms/step - accuracy: 0.3701 - loss: 1.7077 - val_accuracy: 0.5031 - val_loss: 1.3802
Epoch 2/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 41s 66ms/step - accuracy: 0.5175 - loss: 1.3508 - val_accuracy: 0.5879 - val_loss: 1.1680
Epoch 3/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 37s 60ms/step - accuracy: 0.5773 - loss: 1.1981 - val_accuracy: 0.6286 - val_loss: 1.0528
Epoch 4/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 37s 59ms/step - accuracy: 0.6163 - loss: 1.0948 - val_accuracy: 0.6474 - val_loss: 0.9984
Epoch 5/5
625/625 ━━━━━━━━━━━━━━━━━━━━ 40s 64ms/step - accuracy: 0.6474 - loss: 1.0050 - val_accuracy: 0.6583 - val_loss: 0.9717
Test Accuracy: 0.656499981880188
Enter image path: /content/images.jpg
Predicted Class: Bird
Confidence: 46.513668
```



CONCLUSION :-

The experiment successfully implemented a Convolutional Neural Network for image classification using the CIFAR-10 dataset. The CNN learned useful image features through convolution and pooling layers and classified the input images into different object categories.